

1. Prérequis et Installation

Avant d'exécuter le code Python, il faut installer les bibliothèques nécessaires dans votre environnement virtuel (`.venv`).

Ouvrez un terminal et exécutez la commande suivante :

```
pip install sqlalchemy pyodbc sqlalchemy-access
```

Rôle des dépendances :

- **sqlalchemy** : L'ORM et le moteur SQL principal de Python.
- **pyodbc** : Le pilote qui permet à Python de communiquer avec l'interface ODBC de Windows.
- **sqlalchemy-access** : Le module d'extension (dialecte) qui adapte SQLAlchemy aux spécificités de Microsoft Access.

Remarque importante : L'architecture de votre Python (32 bits ou 64 bits) doit correspondre exactement à celle de Microsoft Office / Microsoft Access installée sur votre machine.

2. Code complet d'initialisation et de requête

Voici le script complet pour se connecter et interroger la base :

```
import sqlalchemy
import sqlalchemy.orm

# 1. Chemin absolu vers le fichier Access (.accdb)
db_path = r"D:\K\Documents\Database1.accdb"

# 2. Nom exact du driver ODBC Windows pour Access
odbc_driver = "Microsoft Access Driver (*.mdb, *.accdb)"

# 3. Construction de la chaîne de connexion ODBC
connection_string = (
    f"DRIVER={{odbc_driver}};"
    f"DBQ={db_path};"
    "ReadOnly=1;"          # Mode lecture seule (évite les conflits d'accès
                           # concurrentiel)
    "ExtendedAnsiSQL=1;"  # Active la compatibilité SQL standard (support des
                           # guillemets, etc.)
)

# 4. Création sécurisée de l'URL SQLAlchemy
connection_url = sqlalchemy.URL.create(
    "access+pyodbc",
    query={"odbc_connect": connection_string}
)
```

```
# 5. Création du moteur (Engine) et de la fabrique de sessions (sessionmaker)
engine = sqlalchemy.create_engine(url=connection_url)
session_maker = sqlalchemy.orm.sessionmaker(bind=engine)

# 6. Exécution de la requête SQL dans un bloc contextuel (with)
with session_maker() as session:
    query = sqlalchemy.text("SELECT * FROM Personnes WHERE nom LIKE 'LY'")
    data = session.execute(query).all()
    print(data)
```

3. Explications détaillées du code

A. La chaîne de connexion (`connection_string`)

- **`DRIVER={...}`** : Spécifie le pilote ODBC utilisé par Windows. Les trois accolades `{{{ ... }}}` permettent d'échapper les caractères dans une *f-string*.
- **`DBQ=...`** : Indique le chemin physique du fichier `.accdb`. Le préfixe `r"""` (raw string) évite que les antislashes `\` ne soient interprétés comme des caractères d'échappement.
- **`ReadOnly=1`** : Permet de lire les données même si le fichier `.accdb` est actuellement ouvert dans le logiciel Microsoft Access.
- **`ExtendedAnsiSQL=1`** : Force le moteur Access à utiliser la syntaxe SQL ANSI standard.

B. Création de l'URL (`sqlalchemy.URL.create`)

Au lieu de manipuler directement des chaînes de texte complexes, on utilise `URL.create()`. Cela permet à SQLAlchemy de gérer automatiquement le codage des caractères spéciaux (espaces, parenthèses) de la chaîne ODBC.

C. La Session et la requête (`session_maker`)

- **`sqlalchemy.orm.sessionmaker(bind=engine)`** : Prépare la fabrique de sessions associées à notre base Access.
- **`with session_maker() as session:`** : Ouvre une session de travail et garantit sa fermeture propre après exécution, même en cas d'erreur.
- **`sqlalchemy.text(...)`** : Indique à SQLAlchemy d'exécuter une requête SQL brute.